Smart Signal: Complete Technical Documentation

Machine learning swing-trading system for NSE equities, Fyers paper-trading bot and ATLAS dashboard

Companion document to the Machine Learning (CSE_3148) mini project report

Manipal Institute of Technology, Bengaluru

Arjit Menon, Registration No. 245805448

Version 2 of the system, October 2026

Contents

1 Overview and design goals
2 System architecture and data flow
3 Data: sources, download, cleaning and daily updates
4 Feature engineering (all 57 features)
5 Labels: the trade simulator and the Fyers cost model
6 The model: bagged histogram gradient boosting
7 Training procedure: walk-forward validation, embargo and threshold choice
8 Trade gate and portfolio rules
9 Results in detail
10 Live inference (ml_signals.py)
11 The trading bot (bot.py) and order execution (fyers_execution.py)
12 Fyers API integration: login, tokens and rate limits
13 The ATLAS web application: front end and API
14 The three bot modes: LIVE, SIM and LIVE SIM
15 File reference
16 How to run it: commands and daily routine
17 Troubleshooting
18 Testing and verification
19 Limitations and honest caveats
20 Glossary

1. Overview and design goals

Smart Signal is a complete, locally run trading system. Every day it downloads the latest daily prices for about 190 liquid NSE stocks, computes 57 indicators per stock, asks a machine learning model how likely a short swing trade in each stock is to make money after charges, and buys only the very best setups. Each position is closed automatically at a take-profit level, a stop-loss level or after 15 trading days. Orders go through the Fyers broker API in paper-trading mode (no real money moves unless a single switch is changed by the user), and everything is visible on a custom web dashboard called ATLAS.

The project went through two versions. Version 1 predicted BUY/HOLD/SELL five days ahead for four stocks with 37.3% accuracy and, in live use, bought two stocks and never sold them, because it had no exit logic other than a SELL prediction that never came. Version 2 was rebuilt around five design goals:

1 **Learn what is actually traded.** The label is the result of the exact trade the bot will place (entry, take-profit, stop-loss, time stop), after real Fyers charges, not a generic 'price goes up' label.
2 **Use much more data.** 191 stocks over 19 years (741,980 stock-days) instead of 4 stocks over 5 years.
3 **Validate honestly.** Every reported result comes from a model that had never seen the year it was tested on, with a 30-day embargo between training and test data, and the final years (2022 to 2026) kept as a holdout.
4 **Always exit.** Every position has a pre-computed take-profit and stop-loss and a hard time limit.
5 **Be operable.** Token expiry, broker rate limits, restarts and stale data are handled automatically, and the dashboard shows exactly what the bot is doing.

Headline numbers. On the 2022 to October 2026 holdout the system took 87 trades with an 87.4% win rate after charges, an average of +0.76% net per trade and a profit factor of 2.04. Across all out-of-sample years from 2018 it took 133 trades with an 89.5% win rate, a profit factor of 2.12 and a total profit of Rs. 57,011 at Rs. 50,000 per trade. Section 9 and Section 19 explain how to read these numbers and their limits.

2. System architecture and data flow

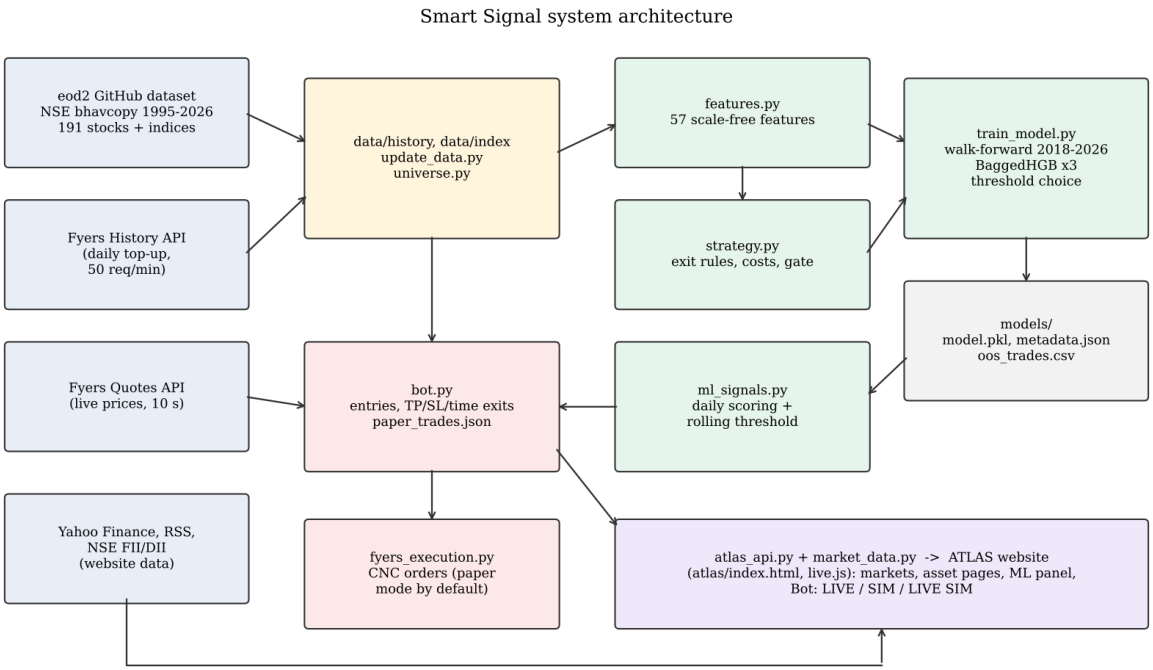


Figure 1. System architecture.

The system has an offline part, run once or occasionally, and an online part that runs every trading day.

2.1 Offline pipeline

1 **Download history** (update_data.py, universe.py): split-adjusted daily OHLCV for 191 stocks, NIFTY 50, India VIX and sector indices from the eod2 dataset on GitHub, topped up from the Fyers History API.

2 **Build the dataset** (train_model.py calling features.py and strategy.py): indicators, market features, cross-sectional features and the triple-barrier label for every stock-day.

3 **Walk-forward evaluation** (train_model.py): one model per test year 2018 to 2026, each trained only on older data.

4 **Threshold choice**: the score quantile that defines a 'buy' is chosen on 2018 to 2021 only.

5 **Final model**: trained on all data and saved to models/model.pkl with models/metadata.json.

6 **Reports** (train_model.py, backtest.py): charts in reports/ and the trade list models/oos_trades.csv.

2.2 Online loop (every trading day)

1 From 08:30 the bot updates the last few days of prices from Fyers (rate-limited) and scores every stock's latest completed daily bar.

2 Scores are compared with the rolling threshold (top 0.3% of the last 250 days of scores) and the gate.

3 From the 09:15 open until 11:00 it buys the qualifying stocks (best 3 per day, at most 10 open positions).

4 Every 10 seconds it fetches quotes for held stocks and sells at take-profit or stop-loss.
5 At 15:20 it sells positions that reached the 15-day time limit.
6 Everything is written to paper_trades.json and shown on ATLAS at http://127.0.0.1:5000.

2.3 Technology stack

Layer	Technology
Language	Python 3.12 (backend, ML), JavaScript ES2020, HTML and CSS (front end)
Data and ML	pandas, NumPy, scikit-learn 1.9 (HistGradientBoostingClassifier), joblib
Web server	Flask (one process serves the bot API and the ATLAS site), Blueprint in atlas_api.py
Broker	fyers-apiv3 Python SDK (quotes, history, orders, login)
Website data	yfinance (global quotes, history, fundamentals), RSS news feeds, NSE public endpoints
Charts	matplotlib for report figures; hand-written SVG and canvas charts in the browser
Operating system	macOS (the .command launchers), also runs on Linux and Windows with python commands

3. Data: sources, download, cleaning and daily updates

3.1 Sources

Source	What it gives	Where it is used
eod2 dataset, github.com/BennyThadikaran/eod2_data	NSE bhavcopy daily OHLCV, adjusted for splits and bonuses, from 1995	Bulk history for training (fetched from raw.githubusercontent.com)
Fyers History API	Daily candles	Cross-check of eod2 and daily top-up of the last days
Fyers Quotes API	Last traded price, change, volume	Live entries and exits; LIVE badge on ATLAS
Fyers charges list	Brokerage, STT, exchange fee, stamp duty, GST, DP fee	strategy.round_trip_cost()
Yahoo Finance via yfinance	Global indices, US stocks, FX, crypto, commodities, fundamentals	ATLAS pages only
RSS feeds and NSE	News headlines, FII/DII flows, sector data	ATLAS pages only

Only eod2 and Fyers data are used for training and trading. Website-only sources never influence the model.

3.2 Universe

universe.py defines the trading universe: 191 liquid NSE stocks (essentially the F&O list), plus NIFTY50 and INDIAVIX. FOCUS_SYMBOLS are the four stocks of version 1 (HDFCBANK, AUBANK, BAJAJFINSV, RELIANCE), which are always shown on the dashboard. fyers_symbol() converts a short name like HDFCBANK to the broker format NSE:HDFCBANK-EQ (indices use NSE:NIFTY50-INDEX and NSE:INDIAVIX-INDEX).

3.3 Cleaning

- **Start in 2007.** Older rows in the source contain unadjusted corporate actions.
- **Segmenting at jumps.** universe.segments() splits a stock's history wherever the price moves more than 35% in one day (demergers, unadjusted actions). Indicators and labels are computed inside each segment separately, so no 200-day average or 15-day label ever crosses an artificial jump. Segments shorter than 300 days are dropped; live scoring uses last_segment().
- **Warm-up.** Rows without a full year of history (needed for SMA 200 and its 20-day slope) are removed.
- **Missing market data.** NIFTY 50 history starts in 2012 and India VIX in 2014. Earlier rows keep NaN for those features; gradient-boosted trees route missing values natively, so no imputation is needed. Because VIX is entirely missing in early years, walk-forward test years start in 2018.
- **Verification.** eod2 and Fyers matched to the paisa on 1,233 overlapping days per stock.

Result: 741,980 stock-days for 191 stocks from 19 October 2007 to 13 October 2026, of which 740,519 have a complete label (the last 15 days cannot be labelled yet).

3.4 Daily updates (update_data.py)

update_from_fyers() downloads the most recent days for every symbol and appends them to data/history/SYMBOL.csv. The Fyers Standard plan allows 50 history requests per minute; exceeding it returns error code -360 and temporarily blocks requests. The updater therefore waits PACE_SECONDS = 1.5 seconds between requests (40 per minute), skips symbols that are already current, retries failed requests, and pauses 62 seconds when a rate-limit error appears. update_from_github() is a fallback that re-downloads from eod2. A symbol whose latest bar is older than 5 calendar days (STALE_DAYS) is never traded.

4. Feature engineering (all 57 features)

All features are scale-free (percentages, ratios, ranks or bounded oscillators), so one model can learn from a Rs. 100 stock and a Rs. 30,000 stock together. They are computed by features.py with vectorised pandas code, using only data up to and including day t (no look-ahead). The same code runs in training and live.

4.1 Stock features (39)

Group	Features	Definition
Trend distance	ema_gap_pct, dist_ema20, dist_sma20, dist_sma50, dist_sma200	EMA 9 vs EMA 21 gap; close vs EMA 20, SMA 20, 50 and 200, in %
Long trend	sma50_200_gap, sma200_slope	SMA 50 vs SMA 200 in %; % change of SMA 200 over 20 days
Oscillators	rsi, rsi2, rsi5, stoch, williams_r, cci	Wilder RSI over 14, 2 and 5 days; Stochastic %K; Williams %R; Commodity Channel Index
MACD and ADX	macd_pct, macd_hist_pct, adx, di_diff	MACD line and histogram as % of price; ADX 14; +DI minus -DI
Momentum	mom_1, mom_3, mom_5, mom_10, mom_20, mom_60	% return over 1 to 60 days
Bands and volatility	bb_pct_b, bb_width, atr_pct, volatility_10, vol_ratio_10_60	Bollinger %B and width; ATR 14 as % of price; 10-day stdev of returns; 10 vs 60-day volatility
Volume	volume_ratio, vwap_gap_pct, obv_roc	Volume vs 20-day average; close vs 20-day rolling VWAP; OBV rate of change
Candle and short-term	ibs, gap_pct, down_streak, dist_low5, dist_high5, zscore_20	Internal bar strength (close-low)/(high-low); opening gap; consecutive down days; distance from 5-day low and high; 20-day z-score
52-week	pct_from_52w_high, pct_from_52w_low	Distance from the 252-day high and low

4.2 Market features (9)

mkt_mom_5, mkt_mom_20 (NIFTY 50 returns), mkt_dist_sma200 (NIFTY vs its 200-day average), mkt_rsi, mkt_rsi2, vix (India VIX level), vix_chg_5 (5-day VIX change), rel_str_20 and rel_str_60 (stock return minus NIFTY return over 20 and 60 days). These tell the model whether the whole market is calm or stressed.

4.3 Cross-sectional features (9)

add_cross_section() ranks every stock against all other stocks on the same day: cs_mom_5, cs_rsi2, cs_mom_20, cs_dist_sma200, cs_zscore_20, cs_volume_ratio and cs_atr_pct are percentile ranks between 0 and 1. breadth_50 is the share of stocks above their 50-day average and breadth_down5 the share down over 5 days. A stock that is oversold while most of the market is fine behaves differently from one falling with everything else; these features capture that. Live, the same ranks are computed across the whole universe on the latest day, which is why the bot always scores all stocks together.

4.4 ATLAS indicators

The ATLAS front end shows SMA 20, SMA 50, SMA 200 and EMA 20 on its charts. These were added to the model as dist_sma20, dist_sma50, dist_sma200 and dist_ema20 so that what the user sees on a chart is also what the model uses.

5. Labels: the trade simulator and the Fyers cost model

5.1 Exit rules (strategy.py)

```
TP_ATR = 0.75     take-profit = entry x (1 + clip(0.75 x ATR%, 0.8%, 6%))
SL_ATR = 4.0      stop-loss   = entry x (1 - clip(4.0  x ATR%, 2%, 15%))
MAX_HOLD = 15     trading days, then exit at that day's close
```

ATR% is the 14-day Average True Range divided by the close on the signal day. Using ATR makes the levels adapt to each stock's normal daily movement: a quiet bank gets tighter levels than a volatile metal stock. The target is close and the stop is far, which gives a naturally high win rate; the model's job is to avoid the setups where the rare large loss is likely.

5.2 Simulating one trade per stock-day

simulate_outcomes() runs, for every row t of a stock, a trade bought at the open of day t+1, then walks forward day by day:

- If the day opens beyond a level (a gap), the trade exits at the open price, which can be better or worse than the level.
- If the day's high reaches the take-profit and the low does not reach the stop, the exit is at the take-profit.
- If the low reaches the stop, the exit is at the stop. If both are touched on the same day, the stop is assumed to have come first (pessimistic, since daily bars do not show the order).
- If neither level is hit by day 15, the exit is at day 15's close.

The output per row is the gross return, the net return after costs, the exit reason (target, stop, time) and the holding period. The label is 1 if the net return is above zero.

5.3 Cost model

round_trip_cost(v) returns the total cost of a buy and a sell of value v rupees in delivery (CNC), as a fraction:

Charge	Rule	At Rs. 50,000
Brokerage	min(Rs. 20, 0.3%) per order, plus 18% GST, two orders	Rs. 47.20
STT	0.1% on buy and 0.1% on sell	Rs. 100.00
Exchange charge	0.0030699% per side, plus 18% GST	Rs. 3.62
Stamp duty	0.015% on buy	Rs. 7.50
DP charge	Rs. 12.50 plus GST per sell	Rs. 14.75
Slippage (assumed)	0.05% per side	Rs. 50.00
Total		**Rs. 223.07 = 0.446%**

Because brokerage and DP fees are fixed in rupees, smaller trades pay a larger percentage: 1.33% at Rs. 5,000 and 0.38% at Rs. 1,00,000. This is why the strategy is sized at Rs. 50,000 per trade.

5.4 Base rates

Buying every stock every day with these rules wins 77.1% of the time but loses money on average (-0.13% per trade), because the average loss is about four times the average win. A useful model must lift the win rate to well above 85% to make the expected value clearly positive. Break-even win rate is roughly 80% for these payoffs.

6. The model: bagged histogram gradient boosting

The classifier is ensemble.BaggedHGB: three sklearn HistGradientBoostingClassifier models trained with different random seeds and max_features = 0.6 (each tree split sees a random 60% of the features). Their predicted probabilities are averaged.

```
HistGradientBoostingClassifier(
    max_iter=300, learning_rate=0.05, max_leaf_nodes=31,
    min_samples_leaf=400, l2_regularization=1.0, max_features=0.6,
    random_state=seed)            # seeds 0, 1, 2; probabilities averaged
```

Why this model. Gradient-boosted trees are the standard strong baseline for tabular data: they capture non-linear interactions between indicators (for example 'RSI-2 is very low AND the 200-day trend is rising'), are insensitive to feature scaling and handle missing values without imputation. The histogram variant bins each feature into 255 buckets, which makes training on 740,000 rows take seconds to minutes rather than hours. min_samples_leaf = 400 stops trees from memorising small groups of days, which matters because financial labels are very noisy. Bagging three seeds reduces variance in the extreme high-score tail, which is the only part of the score distribution that is traded.

Alternatives considered. Version 1 compared logistic regression, random forest, extra trees and gradient boosting; boosting was best. In version 2, training only on rows that pass the gate, and plain (unbagged) models, both gave a lower out-of-sample profit factor and were rejected.

7. Training procedure: walk-forward validation, embargo and threshold choice

7.1 Why not a random train/test split

Stock rows close in time are strongly related, and each label looks up to 15 days into the future. A random split would put almost identical rows in training and test sets and leak future information, giving falsely high scores. The project therefore uses walk-forward (time-ordered) validation.

7.2 Walk-forward folds

```
for Y in 2018 .. 2026:
    train = rows with date <= 1 Jan Y - 30 days      # 30-day embargo > 15-day label horizon
    test  = rows with date in year Y
    model_Y = BaggedHGB().fit(train)
    scores[Y] = model_Y.predict_proba(test)
```

Every one of the 392,537 out-of-sample stock-days is therefore scored by a model that never saw its year.

7.3 Rolling quantile threshold

The average level of predicted probabilities drifts between market regimes. A fixed probability cut-off chosen on 2018 to 2021 produced only 3 trades in 2022 to 2026. Instead, a stock-day is a candidate when its score is above the q-quantile of all scores in the previous 250 trading days (rolling_threshold() in train_model.py). This threshold uses only past scores, so it is fully causal and adapts to drift.

choose_quantile() tries q in {0.97, 0.98, 0.99, 0.995, 0.997} on the 2018 to 2021 out-of-sample trades only and picks the most active setting that reaches a 92% win rate (TARGET_WIN_RATE) with a profit factor of at least 1.5 and at least 30 trades. The target is set above 85% because walk-forward results typically lose a few points from calibration to holdout. The choice was q = 0.997 (the top 0.3% of scores). 2022 to 2026 was not used to choose it.

7.4 Final model and saved files

After evaluation, train_model.py trains one final BaggedHGB on all labelled rows and saves:

File	Contents
models/model.pkl	The fitted ensemble (joblib)
models/metadata.json	Feature list, quantile 0.997, current threshold (0.9195), gate, exit rules, holdout statistics, data date
models/oos_trades.csv	All 133 out-of-sample trades: symbol, dates, prices, probability, gross and net return, exit reason
models/recent_scores.csv	The last 250 days of scores, which seed the live rolling threshold
reports/*.png	Walk-forward charts, confidence bands, equity curve, backtest

python train_model.py --final-only retrains only the final model, skipping the walk-forward (useful after a data update).

8. Trade gate and portfolio rules

A losing-trade analysis showed losses clustered in stocks in long downtrends and in panicky markets. strategy.passes_gate() applies four rules after the model has scored a row:

```
GATE = {"sma200_slope_min": 0.0,     # 200-day average rising
        "dist_sma200_min": -10.0,  # price not more than 10% below it
        "vix_max": 30.0,           # India VIX at most 30
        "atr_pct_max": 4.0}        # daily range under 4% of price
```

The model is trained on all rows and the gate filters its output; training only inside the gate was worse. Portfolio rules, identical in backtest and live: one position per stock; at most 3 new trades per day (CAP_PER_DAY, highest scores first); at most 10 open positions (MAX_OPEN); Rs. 50,000 per trade, so up to Rs. 5 lakh can be deployed at once.

9. Results in detail

Period	Trades	Win rate	Avg net	Profit factor	P&L (Rs.)
2018 to 2021 (threshold chosen here)	46	93.5%	+1.04%	2.27	24,012
2022 to Oct 2026 holdout	**87**	**87.4%**	**+0.76%**	**2.04**	**32,999**
All out-of-sample 2018 to 2026	133	89.5%	+0.86%	2.12	57,011

Year	2018	2019	2020	2021	2022	2023	2024	2025	2026
Trades	13	9	2	22	7	12	39	12	17
Win rate	100%	100%	50%	91%	86%	100%	90%	92%	71%

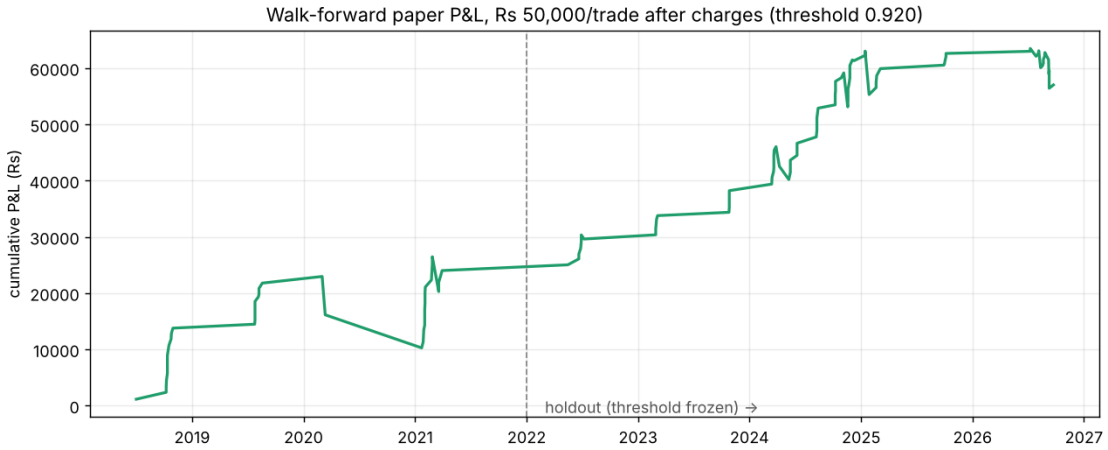


Figure 2. Cumulative out-of-sample P&L at Rs. 50,000 per trade after all charges.

- Exits: 119 take-profit, 9 time stop, 5 stop-loss. Average holding period 4.2 trading days.
- Average winning trade +1.8% net, average losing trade -7.2% net. Profit factor = gross wins / gross losses.
- 95% confidence interval for the 89.5% win rate on 133 trades: about 84% to 95%.
- Overall ROC AUC of the scores is about 0.525: the model ranks ordinary days only slightly better than chance. The edge is in the extreme tail: top-0.3% gated stock-days won 85.2%, and the portfolio rules raise this to 89.5%.
- Cost sensitivity: profit factor 0.97 at Rs. 5,000 per trade (loses money), 1.44 at Rs. 10,000, 2.12 at Rs. 50,000.
- Activity: about 16 trades a year. Rs. 57,011 over 8.2 years on up to Rs. 5 lakh reserved is about 1.3% a year, far below NIFTY 50 buy-and-hold (about 10% a year). The high win rate does not by itself mean high returns.

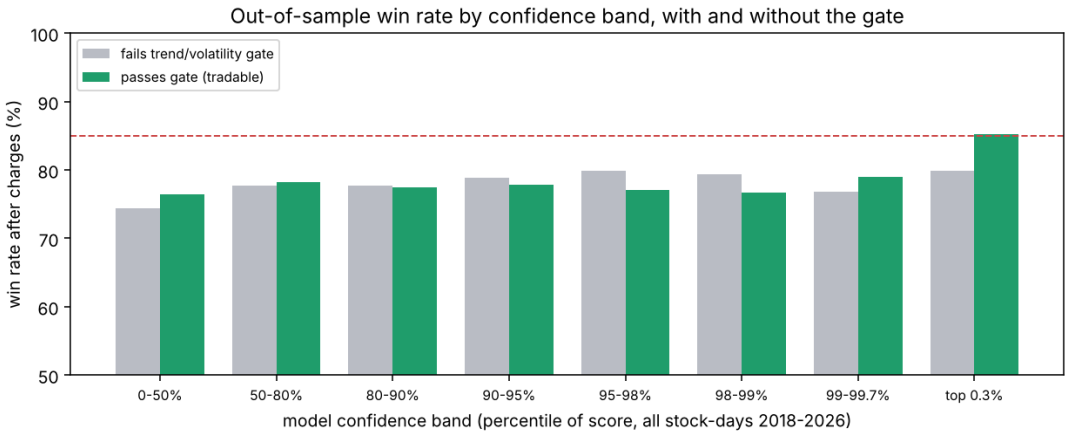


Figure 3. Out-of-sample win rate by model confidence band, inside and outside the gate.

10. Live inference (ml_signals.py)

ml_signals.Model loads model.pkl and metadata.json and provides:

- **completed_bars()**: drops today's partial candle so only finished daily bars are scored (the model was trained on complete bars).
- **daily_candidates(symbols, today)**: loads each stock's last segment, builds all 57 features including the cross-section across the whole universe, scores the latest bar, applies the gate and returns candidates sorted by score. Stocks whose last bar is more than 5 days old are excluded.
- **current_threshold()**: the 0.997 quantile of the last 250 days of scores, from models/recent_scores.csv plus every day the live bot has logged to data/score_history.csv (log_scores()). The threshold therefore keeps rolling forward in live use exactly as in the backtest.
- **plan()**: for a candidate, the entry reference, take-profit and stop-loss prices from strategy.exit_levels().
- **universe_snapshot()**: scores for every stock for the ATLAS model panel (display only, staleness allowed).

11. The trading bot (bot.py) and order execution (fyers_execution.py)

11.1 Daily timeline

Time (IST)	Action
08:30 onward	morning_routine() in a background thread: update data from Fyers, score all stocks, log scores, pick candidates. Retries every 5 minutes on failure.
09:15 to 11:00	Buy qualifying candidates (best 3 per day, max 10 open). ENTRY_CUTOFF = 11:00 keeps entries near the open, as in the label.
Every 10 s	Fetch quotes (batched 50 symbols per call) for held stocks; sell when price reaches take-profit or stop-loss.
15:20	TIME_STOP_AT: sell positions held for 15 trading days.
Always	Serve the ATLAS website and API on port 5000; save state to paper_trades.json after every change.

11.2 Journal and restarts

paper_trades.json stores open positions (symbol, quantity, entry price and date, take-profit, stop-loss, model probability), closed trades with P&L and reason, and the order log. On restart the bot reloads it, so positions are never forgotten, which was a version 1 failure. Output is line-buffered so the Terminal and bot_output.log show events immediately.

11.3 Execution and safety

Setting	Value	Meaning
LIVE_TRADING	False	All orders are paper trades recorded locally. Only the user may change this.
PAPER_POSITION_VALUE	Rs. 50,000	Size of each paper trade (matches the cost model)
PAPER_DAILY_LOSS_LIMIT	Rs. 25,000	Kill-switch: no new trades after this daily loss
MAX_POSITION_VALUE	Rs. 5,000	Size per trade if live trading is ever turned on
DAILY_LOSS_LIMIT	Rs. 2,000	Live kill-switch
MAX_TRADES_PER_DAY	40	Safety net against a runaway bug
PRODUCT_TYPE	CNC	Delivery orders, because positions are held for days (intraday orders would be squared off)

place_order(symbol, side, price, qty) either records a paper fill or, only if LIVE_TRADING is True, sends a market order through Fyers. Note that the live size of Rs. 5,000 is where the backtest shows the edge is lost to fixed charges; any move to real money should be decided carefully and after more forward testing.

12. Fyers API integration: login, tokens and rate limits

- **Login.** auth.py prints the Fyers login URL; after logging in, the browser is redirected to a page whose URL contains an auth code. get_token.py exchanges it for an access token and saves it to fyers_secrets.json together with the app id. This file is private and must never be shared or uploaded.
- **Daily expiry.** Fyers tokens expire every day. start_today.command and relogin_and_retrain.command run the login flow. bot.py checks the modification time of fyers_secrets.json and hot-reloads the token without a restart.
- **Quotes.** Up to 50 symbols per request; the bot batches accordingly.
- **History.** 50 requests per minute on the Standard plan; error -360 when exceeded. Handled by pacing (Section 3.4).
- **Invalid symbols.** A symbol Fyers does not recognise (ICICIPRULI in the current list) is skipped and logged.

13. The ATLAS web application: front end and API

13.1 Front end

atlas/index.html is the ATLAS single-page site (home, markets, stocks, indices, forex, commodities, crypto, news, India page, screener and per-asset pages). atlas/live.js replaces its demo data with real data from the backend, adds the model panel to each NSE stock page and implements the bot section with its three modes. Routing uses the URL hash (#/bot, #/livesim, #/asset/...). A LIVE DATA badge shows the data source and time.

13.2 API endpoints (atlas_api.py)

Endpoint	Returns
/ , /live.js , /classic	ATLAS page, its live-data script, and the older dashboard
/api/health	Server, model and data status
/api/quotes	Quotes for the ATLAS universe; Fyers live prices overlay Yahoo for NSE stocks the bot holds
/api/history?symbol=&range=	Price history for charts
/api/asset?symbol=	The model panel: probability, gate checks, threshold, entry/target/stop plan
/api/fundamentals?symbol=	Valuation and company data
/api/india	Sector indices, VIX, breadth, FII/DII flows
/api/commodities , /api/news	Commodity prices; market and stock news
/api/bot/state	LIVE: positions, orders, today's P&L, signals, last scan
/api/bot/sim	All out-of-sample trades with real daily price paths and summary statistics
/api/bot/replay , /api/bot/replay/speed?x=	SIM: real-data replay state; speed 1, 2, 4 or 15
/api/bot/livesim , /api/bot/livesim/speed?x=	LIVE SIM: simulated market and bot state; speed 1, 2, 4 or 10

13.3 Robustness

market_data.py wraps every outside call in a TTL cache (quotes 60 s, news 90 s) and a 5-minute failure back-off per source, so a slow or failing website never blocks the page. yfinance downloads use 4 threads. The server raises the process open-file limit to 10,240 at start-up, which fixed 'Too many open files' errors on macOS.

14. The three bot modes: LIVE, SIM and LIVE SIM

Mode	Prices	Trades	What it proves
LIVE	Real NSE prices now (Fyers)	The real model's decisions today (paper orders)	The system works end to end on the real market
SIM	Real NSE prices from 2018 to 2026	The 133 real out-of-sample trades, replayed in real time	The model's measured win rate and profit
LIVE SIM	Simulated prices generated live, 24/7	A dip-in-uptrend rule with the strategy's gate and exits	The bot can read charts and trade on its own when NSE is closed

14.1 SIM: replay.py

The Replay engine loads models/oos_trades.csv and the stocks' daily history. It builds a schedule of trading days on which the bot held or opened a position (idle days are skipped), holdout years first. Each day lasts DAY_SECONDS = 45 seconds of wall-clock time with TICKS = 30 price updates. The intraday path is interpolated through the day's real open, high, low and close (high first on a take-profit day, low first on a stop day, plus small noise, clipped to the real range). BUY orders fire at tick 0 at the real entry price; a SELL fires when the path reaches the trade's real exit price, or at the close on a time-stop day. P&L uses the trade's real net return. Progress is saved in replay_state.json so the replay resumes after a restart, and it loops forever.

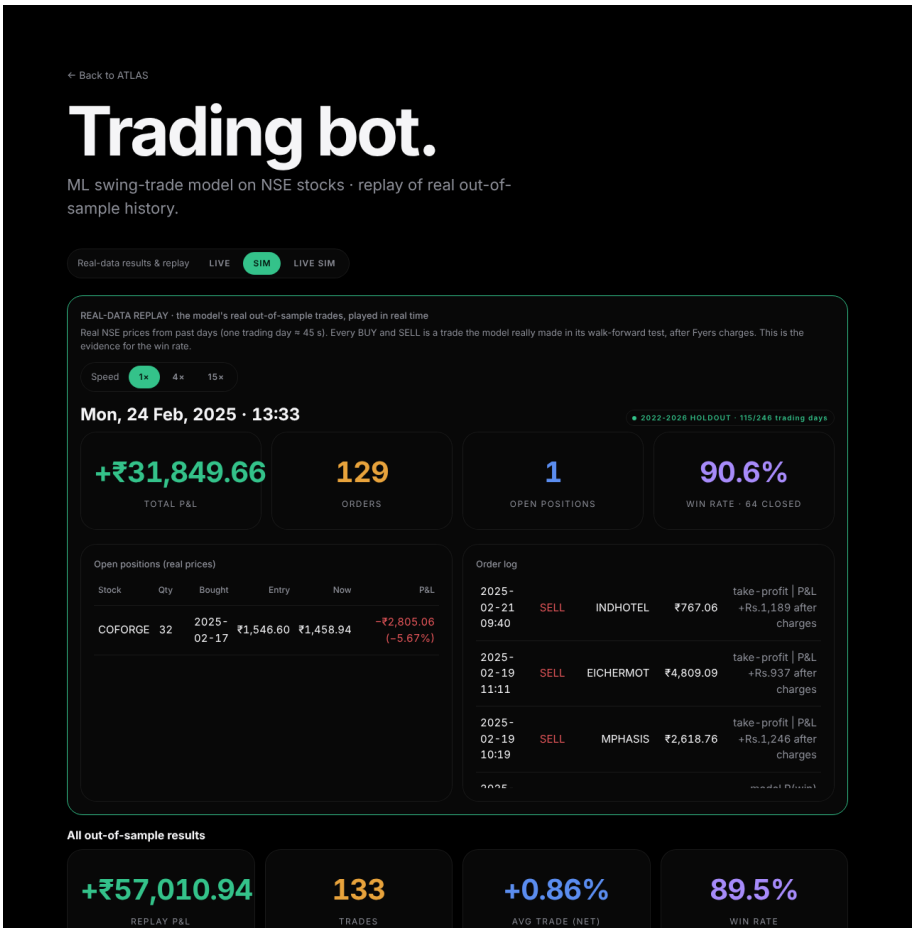


Figure 4. SIM tab.

14.2 LIVE SIM: sim_market.py

Market model. Eight stocks start at their latest real closing price and then evolve without any past data. Each bar represents 5 minutes and is generated from 5 one-minute sub-steps:

```
regime  : up / flat / down, switches with 1% probability per bar (drift +0.025%, 0, -0.02% per bar)
vol     : volatility clusters, log-random walk between 0.15% and 1% per bar
revert  : 4% pull per bar towards a slowly drifting fair value
jumps   : 0.2% chance per minute of a news jump with 1.2% standard deviation
bar     : open, high, low, close from the sub-steps; 75 bars = one 09:15-15:30 session
```

260 warm-up bars are generated at start so that the 200-bar indicators exist from the first second. One bar is produced per real second (BAR_SECONDS = 1), faster with the speed buttons.

Bot logic per bar and stock. indicators() computes RSI 14, RSI 2, SMA 50, SMA 200 and its slope over 20 bars, distance from SMA 200, ATR% and the 20-bar z-score, with the same formulas as features.py. If the stock is held, the bar's low and high are checked against the stop-loss and take-profit (stop first if both, gaps fill at the open) and a time stop applies after 30 bars. If not held, the bot buys at the close when the gate passes (SMA 200 slope above 0, price above -10% of SMA 200, ATR% below 4) and a sharp dip appears (RSI 2 below 5 and z-score below -2), with at most 4 open positions. Take-profit and stop-loss come from strategy.exit_levels(); each trade is Rs. 50,000 and pays strategy.round_trip_cost().

Measured behaviour. Five independent runs of 20 simulated days (487 closed trades): 81.5% win rate, average -Rs. 27 per trade, i.e. about break-even, with individual runs between 73% and 86%. The market was not tuned to make the bot look good. LIVE SIM uses a simple rule on synthetic intraday bars, not the trained daily model, so its numbers must not be quoted as the model's performance; the SIM tab is the evidence.

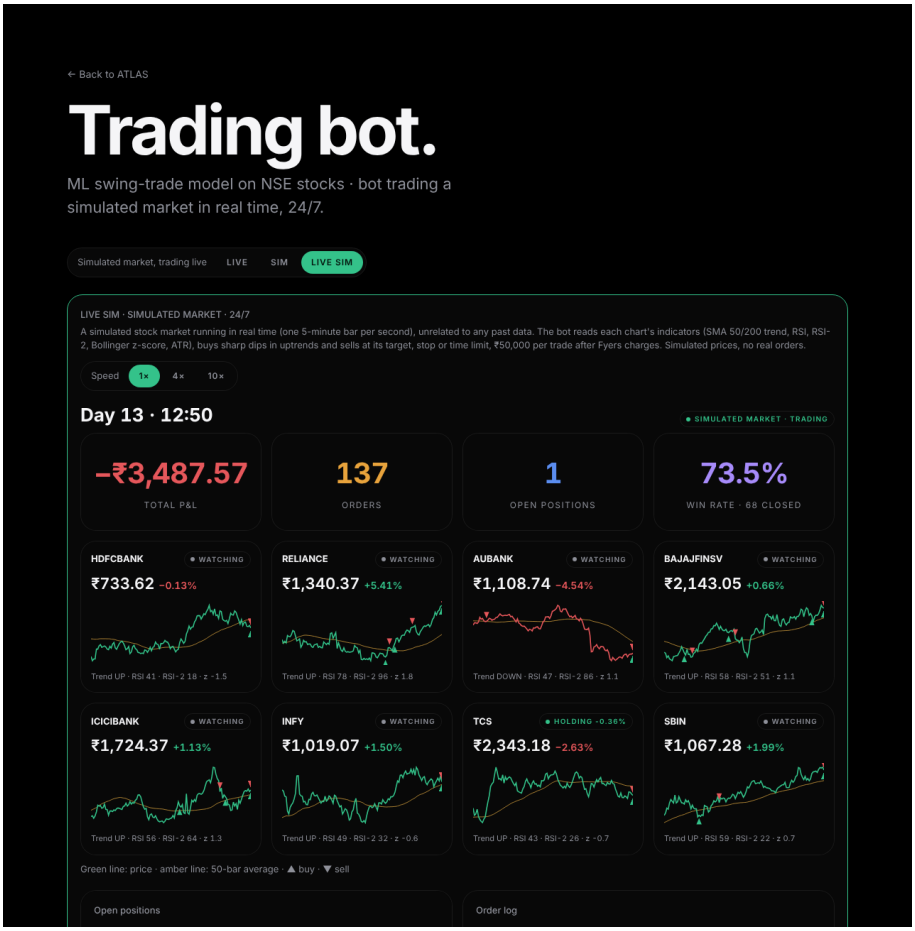


Figure 5. LIVE SIM tab: live simulated charts with buy (up) and sell (down) markers.

15. File reference

File	Purpose
config.py	Paths and general settings
universe.py	Symbol lists, history loading, segmenting at jumps, Fyers symbol names
update_data.py	Daily data top-up from Fyers (paced) or GitHub
fetch_data.py, fetch_data_fyers.py	Version 1 data downloaders (kept for reference)
features.py	All indicators and the 57 model features
strategy.py	Exit rules, cost model, trade simulator, gate
ensemble.py	BaggedHGB classifier
train_model.py	Dataset, walk-forward, threshold, final model, report figures
backtest.py	Backtest summary, cost sensitivity, NIFTY comparison, backtest_v2.png
predict_trend.py	Command-line scoring of stocks with the current model
ml_signals.py	Live scoring, rolling threshold, trade plans
bot.py	The trading bot and the web server entry point
fyers_execution.py	Paper and live order placement, limits, LIVE_TRADING switch
auth.py, get_token.py	Fyers login and token storage
market_data.py	Website data (Yahoo, news, NSE) with caching
atlas_api.py	Flask Blueprint: all ATLAS pages and /api endpoints
atlas_server.py	Runs the website without the bot
replay.py	SIM engine (real out-of-sample replay)
sim_market.py	LIVE SIM engine (simulated 24/7 market)
atlas/index.html, atlas/live.js	Front end
dashboard.html	Older dashboard at /classic
generate_demo_data.py	Version 1 random demo data (no longer used by the bot)
data/history/*.csv, data/score_history.csv	Price history; logged daily scores
models/	model.pkl, metadata.json, oos_trades.csv, recent_scores.csv
paper_trades.json, replay_state.json	Bot journal; SIM progress
fyers_secrets.json	Private token file (never share). fyers_secrets.example.json shows the format.
ML_WALKTHROUGH.md, README.txt	Written guides; ML_WALKTHROUGH_v1.md and backup_v1_2026-10-23/ keep version 1

16. How to run it: commands and daily routine

Double-click (macOS)	What it does
setup_and_login.command	One-time: rebuild the Python virtual environment, install requirements, first Fyers login
start_today.command	Daily: stop any old bot, Fyers login, start the bot and website
restart_clean.command / run_bot.command	Restart the bot and website without logging in again; opens #/bot
open_atlas.command	Website only (no trading)
OPEN_LIVE_SIM.html	Opens ATLAS directly on LIVE SIM (the bot must be running)
relogin_and_retrain.command	Login, update ~190 stocks, full walk-forward retrain, backtest report
run_backtest.command	Backtest report only

Equivalent commands in a terminal, from the project folder with the virtual environment active:

```
source venv/bin/activate
pip install -r requirements_ml.txt
python auth.py && python get_token.py     # daily login
python bot.py                             # bot + website at http://127.0.0.1:5000
python update_data.py --days 120          # data top-up
python train_model.py                     # full walk-forward retrain
python train_model.py --final-only        # retrain final model only
python backtest.py                        # report
```

Demo when the market is closed: start the bot, open ATLAS, select LIVE SIM on the home page or the Bot page (it is selected automatically when NSE is closed), and use the speed buttons. Then switch to SIM to show the real out-of-sample evidence.

17. Troubleshooting

Symptom	Cause and fix
White page in the browser	The page loaded before the server was up. Wait for the Terminal to show requests, then reload.
Internal Server Error / 'Too many open files'	Fixed by the raised file limit and back-off; restart with restart_clean.command if it persists.
Bot says token invalid or expired	Run start_today.command (daily Fyers login). The bot picks up the new token automatically.
Error -360 during update	Fyers rate limit; the updater waits 62 s and continues. Do not run two updaters at once.
No trades today	Normal: the strategy buys only the top 0.3% of setups that pass the gate. The Bot page lists the best scores and why they did not qualify.
'history not current' for a symbol	Fyers has no data for that symbol (ICICIPRULI); it is skipped.
Port 5000 already in use	An old bot is running; restart_clean.command stops it first. On macOS, AirPlay Receiver can also use port 5000.

18. Testing and verification

- Data: eod2 versus Fyers comparison on overlapping days; checks for split jumps (segmenting).
- No look-ahead: features use data up to day t, labels start at t+1, a 30-day embargo separates training and test.
- End-to-end bot tests with a fake Fyers client: entries, take-profit, stop-loss, time stop, restart with saved positions, token reload and rate-limit handling (ratelimit_test).
- Front-end tests with Playwright in headless Chromium against a test server with fake Yahoo data, including screenshots of every page and checks for JavaScript errors.
- LIVE SIM: headless multi-seed runs of the simulated market to measure its win rate and P&L (Section 14.2).

19. Limitations and honest caveats

- Not every year reaches 85% (2020: 1 of 2; 2026 so far: 12 of 17, 70.6%).
- Overall AUC is about 0.53; the edge lives in a small number of very selective trades, so results have wide uncertainty.
- Low activity and low total return relative to the capital reserved; NIFTY buy-and-hold earned more over the period.
- Survivorship bias: the universe is today's large stocks.
- Exit rules, gate and threshold target were explored on the same history; the holdout is strong but not perfect evidence.
- Fills at the open and at exact levels are assumed; real slippage may be larger on volatile days.
- At small trade sizes (Rs. 5,000, the current live setting) fixed charges remove the edge.
- LIVE SIM is a demonstration of the trading loop in a synthetic market, not evidence of model accuracy.
- This is an academic project, not investment advice; LIVE_TRADING remains off.

20. Glossary

Term	Meaning
ATR	Average True Range: average daily price range including gaps (Wilder, 14 days)
RSI-2	Relative Strength Index over 2 days; below 5 means a very sharp short-term drop
SMA / EMA	Simple / exponential moving average
Z-score	How many standard deviations the price is from its 20-day mean
Triple-barrier label	Outcome of a trade with an upper barrier (target), lower barrier (stop) and time barrier
Walk-forward	Train on the past, test on the next period, move forward and repeat
Embargo	Gap between training and test data so overlapping labels cannot leak information
Profit factor	Sum of winning trades divided by the sum of losing trades; above 1 means profitable
ROC AUC	Probability that a random winning row scores higher than a random losing row (0.5 = chance)
CNC	Fyers delivery product type for positions held overnight
Paper trading	Simulated orders recorded locally with real prices; no money moves